

DALUZED

SISTEMA DE GESTIÓN DE INVENTARIO

Informe Completo de Pruebas Unitarias y de Integración

Versión del plan: 1.0 — Mayo 2026

Fecha del informe: 14 de Junio de 2026

Repositorio: PN-daluzed / rama main

✓ 346 TESTS PASAN

✓ 0 FALLOS

Backend Django · 158 tests

Frontend React · 188 tests

Plan original · 107 casos

1. Resumen Ejecutivo

El presente informe documenta el estado completo de las pruebas del sistema **Daluzed** — sistema de gestión de inventario para repostería — comparando los casos definidos en el *Plan de Pruebas TDD v1.0* contra la implementación actual en la rama `main`.

346	346	0	+48%
TESTS TOTALES	TESTS QUE PASAN	TESTS FALLIDOS	SOBRE EL PLAN
Backend + Frontend	100 % de éxito	Sin errores	102 → 158 backend

Capa	Tests implementados	Pasan	Fallan	Estado
Backend (Django + DRF)	158	158	0	✅ OK
Frontend (React + Vitest)	188	188	0	✅ OK
TOTAL	346	346	0	✅ PASAN TODOS

Conclusión anticipada: El proyecto cumple y supera el plan de pruebas TDD. Todos los IDs de prueba definidos en el plan (AUT-001..015, CAT-001..018, INV-001..012, REC-001..010, PROD-001..014, ALR-001..016, IND-001..011, AUD-001..006) están implementados y pasan sin errores. Se añadieron 56 tests backend adicionales que cubren funcionalidad no contemplada en el plan original.

2. Backend — Análisis por Módulo

2.1 Comparativa Plan vs. Implementado

Módulo	Casos en Plan	Tests Implementados	Delta	Estado
AUT — Autenticación y Autorización	15	33	+18	✅ Excede
CAT — Catálogo Maestro	18	25	+7	✅ Excede
INV — Inventario	12	30	+18	✅ Excede
REC — Recepción	10	23	+13	✅ Excede

Módulo	Casos en Plan	Tests Implementados	Delta	Estado
PROD — Producción y Despacho	14	14	—	✅ Cumple
ALR — Alertas y Notificaciones	16	16	—	✅ Cumple
IND — Indicadores y Reportes	11	11	—	✅ Cumple
AUD — Auditoría y Trazabilidad	6	6	—	✅ Cumple
TOTAL	102 *	158	+56	✅ +55%

* El plan lista 107 en el título; las tablas de IDs suman 102 casos de backend. Los 5 restantes corresponden al módulo de Frontend (sección 11 del plan).

2.2 Distribución de Archivos de Test

Archivo	Tests	IDs del Plan	Notas
<code>authentication/tests/test_autenticacion.py</code>	15	AUT-001 .. 015	Exactos del plan
<code>authentication/tests/test_usuarios.py</code>	12	—	Extra CRUD de usuarios vía API, RBAC, desactivación
<code>authentication/test_cambio_contrasena.py</code>	6	—	Extra Endpoint cambio de contraseña
<code>catalogo/tests/test_catalogo.py</code>	25	CAT-001 .. 018	Plan + 7 casos extra de proveedores y presentaciones
<code>inventario/tests/test_inventario.py</code>	14	INV-001 .. 012	Plan + 2 extra (devoluciones, descartes extendidos)
<code>inventario/tests/test_kardex.py</code>	8	—	Extra Saldo acumulado, filtros, FEFO
<code>inventario/tests/test_zonas.py</code>	8	—	Extra CRUD zonas de bodega, permisos por rol
<code>recepcion/tests/test_recepcion.py</code>	12	REC-001 .. 010	Plan + 2 extra
<code>recepcion/tests/test_parcial.py</code>	6	—	Extra Recepciones parciales, estado PARCIAL/RECIBIDA
<code>recepcion/tests/test_pdf.py</code>	5	—	Extra Generación PDF, content-type, 404
<code>produccion/tests/test_produccion.py</code>	14	PROD-001 .. 014	Exactos del plan

Archivo	Tests	IDs del Plan	Notas
<code>alertas/tests/test_alertas.py</code>	16	ALR-001 .. 016	Exactos del plan
<code>indicadores/tests/test_indicadores.py</code>	11	IND-001 .. 011	Exactos del plan
<code>auditoria/tests/test_auditoria.py</code>	6	AUD-001 .. 006	Exactos del plan
TOTAL	158	14 archivos — Ran 158 tests in 20.266s — OK	

2.3 Cobertura de IDs del Plan

Todos los identificadores definidos en `PLAN_DE_PRUEBAS_TDD.md` están implementados y pasan:

Módulo	Rango de IDs	Cantidad	Cubiertos	Estado
AUT	AUT-001 .. AUT-015	15	15	✓ 100%
CAT	CAT-001 .. CAT-018	18	18	✓ 100%
INV	INV-001 .. INV-012	12	12	✓ 100%
REC	REC-001 .. REC-010	10	10	✓ 100%
PROD	PROD-001 .. PROD-014	14	14	✓ 100%
ALR	ALR-001 .. ALR-016	16	16	✓ 100%
IND	IND-001 .. IND-011	11	11	✓ 100%
AUD	AUD-001 .. AUD-006	6	6	✓ 100%
TOTAL	—	102	102	✓ 100%

2.4 Tests Adicionales Más Allá del Plan

Funcionalidad cubierta que el plan original no contemplaba, aportando robustez extra al sistema:

Área	Tests extra	Qué verifican
Gestión de usuarios (CRUD)	12	Listar, crear, cambio de rol, desactivar, password débil, email duplicado, autoprotección admin
Cambio de contraseña	6	Contraseña actual incorrecta, confirmación no coincide, contraseña igual a la actual, contraseña débil, sin autenticar

Área	Tests extra	Qué verifican
Kardex de inventario	8	Saldo acumulado por movimiento, filtro por tipo, filtro por rango de fechas, acceso sin autenticar
Zonas de bodega	8	CRUD de zonas, permisos diferenciados por rol (INVENTARIO vs. GERENTE), listado filtrado por bodega
Recepciones parciales	6	Primera recepción cambia estado OC a PARCIAL, segunda recepción permitida, estado RECIBIDA al completar, saldo pendiente en serializer
PDF de recepción	5	Requiere autenticación, content-type <code>application/pdf</code> , Content-Disposition con ID, cuerpo no vacío, 404 para ID inexistente

3. Frontend — Análisis de Tests React

3.1 Resumen de Ejecución

Test Files 39 passed (39)
Tests 188 passed (188)
Duration ~12 segundos
Framework Vitest + React Testing Library

3.2 Cobertura de Código (Vitest + c8)

Métrica	Resultado	Umbral RNF-MAN-02	Estado
Statements (sentencias)	68.46%	≥ 70%	⚠ Por debajo
Branches (ramas)	65.85%	≥ 70%	⚠ Por debajo
Functions (funciones)	57.41%	≥ 70%	✖ Debajo
Lines (líneas)	70.15%	≥ 70%	✅ Pasa mínimo

Statements — 68.46%

meta: 70%

Branches — 65.85%

meta: 70%

Functions — 57.41%

meta: 70%

Lines — 70.15%

meta: 70%

3.3 Archivos con Cobertura Crítica Baja

Archivo	Lines%	Functions%	Problema
hooks/inventario/useInventario.js	6.52%	6.06%	Todas las funciones del hook sin cubrir (fetchStock, fetchLotes, etc.)
pages/inventario/DescarteForm.jsx	7.14%	0%	Sin ningún test — archivo completamente descubierto
hooks/alertas/useAlertas.js	20%	17.64%	Solo mock de WebSocket cubierto; resolución y filtrado sin probar
hooks/useApi.js	42.85%	50%	Lógica de loading/error en ramas sin cubrir

Archivo	Lines%	Functions%	Problema
pages/catalogo/MateriaPrimaForm.jsx	36.36%	36.36%	Ramas de modo edición vs. creación sin probar
components/layout/TopBar.jsx	38.46%	50%	Menú de usuario y acción de logout sin cubrir
store/authStore.js	50%	50%	<code>clearAuth</code> y lógica de persistencia sin probar

3.4 Archivos con Alta Cobertura (≥ 85%)

Archivo	Lines%	Functions%
components/ui/Button.jsx	100%	100%
components/ui/Confirm.jsx	100%	100%
components/ui/Modal.jsx	100%	100%
components/RoleGate.jsx	100%	100%
pages/inventario/TrazabilidadModal.jsx	100%	100%
utils/vencimiento.js	100%	100%
components/ui/KeyValueEditor.jsx	100%	100%
pages/inventario/StockPage.jsx	94.73%	92.85%
pages/alertas/AlertasReordenPage.jsx	92.85%	100%
pages/recepcion/DetalleRecepcionPage.jsx	92.30%	71.42%
pages/inventario/KardexPage.jsx	88.88%	75%
pages/admin/BitacoraPage.jsx	93.33%	83.33%

3.5 Distribución de Tests Frontend por Área

Área	Archivos	Tests	Qué cubren
Componentes UI (Button, Table, Select, ConfirmDialog...)	4	~18	Renderizado, interacción, accesibilidad
Componentes de layout y navegación	3	~12	AppLayout, RoleGate, ProtectedRoute
Páginas de inventario	5	~33	Stock, Lotes, Bodegas, TrasladoForm, TrazabilidadModal

Área	Archivos	Tests	Qué cubren
Páginas de recepción	3	~15	Órdenes, NuevaRecepción, DetalleRecepción
Páginas de producción	4	~15	Jornada, Batido, Despachos, Compensatorios
Páginas de catálogo	3	~14	MateriasPrimasPage, MateriaPrimaForm, hook useMateriasPrimas
Páginas de alertas	2	~9	AlertasReorden, ConfiguraciónAlertas
Páginas de administración	2	~12	Bitácora, Usuarios
Store (Zustand)	1	5	alertasStore — WebSocket connect/disconnect/mensajes
Hooks	2	~9	useInactivityLogout, useUsuarios
Utilidades	2	~18	formatApiError (mapeo HTTP), vencimiento (colores/labels)
Páginas globales	4	~24	Login, Dashboard, Perfil, Avatar
TOTAL	39	188	

4. Verificación de Reglas de Negocio Críticas

El plan define 13 reglas de negocio críticas que deben estar cubiertas por pruebas. A continuación se verifica su cobertura:

Regla de Negocio	RF	Test que la verifica	Estado
FEFO en consumo de MP desde Bodega PDP	RF-PROD-02	<code>test_prod_002_sugerencia_fefo_para_ingredientes</code>	✓
FIFO en despacho de PT al PDV	RF-PROD-06	<code>test_prod_007_fifo_despacha_lote_mas_antiguo_primerio</code>	✓
Solo Bodega Principal activa alertas de reorden	RF-INV-02	<code>test_alr_002_stock_bodega_pdp_bajo_reorden_no_genera_alerta</code>	✓
Máximo 2 batidos simultáneos en producción	RF-PROD-04	<code>test_prod_004_maximo_dos_batidos_simultaneos</code>	✓
Recepción SOLO contra orden de pedido previa	RF-REC-01	<code>test_rec_002_recepcion_sin_orden_compra_es_rechazada</code>	✓
Bloqueo 5 intentos fallidos → 1 hora (Django-Axes)	RF-AUT-01	<code>test_aut_004_bloqueo_tras_cinco_intentos_fallidos</code>	✓
Registros inmutables — corrección via compensatorio	RF-PROD-08	<code>test_inv_009_traslado_es_inmutable ,</code> <code>test_rec_006_recepcion_confirmada_es_inmutable</code>	✓
Movimiento compensatorio visible para todos los roles	RF-PROD-08	<code>test_prod_010_compensatorio_visible_para_todos_los_rols</code>	✓
Días mínimos de vencimiento — alerta bloqueante	RF-REC-06	<code>test_rec_008_vida_util_insuficiente_genera_alerta_bloqueante</code>	✓
Estados PT: EN_ESPERA → EN_PUNTO_DE_VENTA (irreversible)	RF-PROD-07	<code>test_prod_008_despacho_es_irreversible</code>	✓

Regla de Negocio	RF	Test que la verifica	Estado
Transacciones atómicas en producción y traslados	RF-PROD-01	<code>test_prod_014_transaccion_atomica_revierte_si_falla_a_mitad</code>	✓
RBAC: 4 roles — Admin, Gerencia, Inventario, Producción	RF-AUT-04	<code>test_aut_010</code> , <code>test_aut_011</code> , <code>test_prod_010</code>	✓
user/role NO se persiste en localStorage	RNF-SEG-01	<code>RoleGate.test.jsx</code> , <code>ProtectedRoute.test.jsx</code>	✓

13 de 13 reglas de negocio críticas cubiertas. Todas las reglas definidas en la sección 1.4 del plan de pruebas tienen al menos un test que las verifica directamente y pasa en la ejecución actual.

5. Brechas Identificadas y Recomendaciones

5.1 Cobertura de Funciones Frontend (Prioridad Alta)

La cobertura de funciones está en **57.41%**, por debajo del umbral del 70% definido en RNF-MAN-02. Los tres archivos que más impacto tienen en esta métrica:

- `hooks/inventario/useInventario.js` — el hook central de inventario prácticamente sin tests (6.06% funciones). Cubrir `fetchStock` , `fetchLotes` , y `fetchTrazabilidad` con mocks de axios elevaría la cobertura de funciones ~4–5 puntos porcentuales globales.
- `pages/inventario/DescarteForm.jsx` — 0% de cobertura. Necesita al menos un test de renderizado, validación de campos y submit del formulario.
- `hooks/alertas/useAlertas.js` — solo el mock de WebSocket cubierto. Las funciones de resolución de alertas y filtrado por tipo están sin probar.

5.2 Tests E2E (Prioridad Media)

El sistema no tiene tests de extremo a extremo (E2E) con Playwright o Cypress. Para un MVP de 10 usuarios concurrentes la suite actual es suficiente, pero se recomienda agregar al menos el flujo crítico completo:

`login` → `crear OC` → `recepción` → `traslado` → `batido` → `despacho`

5.3 WebSocket en Backend (Prioridad Baja)

ALR-006 verifica que el servicio invoca `_enviar_websocket` , pero lo hace con un `MagicMock` . No existe un test que verifique la conexión Channels real (requeriría `ChannelsLiveServerTestCase`). Aceptable para el alcance actual del MVP.

5.4 Tabla Resumen de Brechas

Brecha	Impacto	Prioridad	Acción recomendada
Funciones frontend al 57% (debajo del 70%)	Incumple RNF-MAN-02	Alta	Agregar tests a <code>useInventario.js</code> , <code>DescarteForm.jsx</code> , <code>useAlertas.js</code>
Statements (68.46%) y Branches (65.85%) debajo del 70%	Incumple RNF-MAN-02 parcialmente	Media	Los mismos archivos anteriores: al cubrirlos subirá statements y branches también
Sin tests E2E	Flujos completos sin validar	Media	Implementar 1–2 flujos críticos con Playwright
WebSocket Channels con mock solamente	Bajo	Baja	Usar <code>ChannelsLiveServerTestCase</code> si se migra a Redis Layer

6. Conclusión

El proyecto cumple con el plan de pruebas TDD definido. Todos los 102 IDs de prueba backend del `PLAN_DE_PRUEBAS_TDD.md` están implementados y pasan sin errores. Se superó el objetivo con 56 tests adicionales (+55%). Las 13 reglas de negocio críticas tienen cobertura directa.

Los **346 tests combinados** (158 backend + 188 frontend) corren en menos de 35 segundos en total, proporcionan cobertura de líneas del 70.15% en frontend y cobertura completa de los endpoints de negocio en backend con PostgreSQL real (no SQLite ni mocks de BD).

El único punto de atención es la **cobertura de funciones frontend al 57.41%**, por debajo del umbral del 70% de RNF-MAN-02. Este déficit se concentra en tres archivos (`useInventario.js` , `DescarteForm.jsx` , `useAlertas.js`) y puede resolverse con aproximadamente 15–20 tests adicionales focalizados en esas áreas.